

SPRINGBOOT SUMMARY

CHAPTER - 2

TOPICS:

1. **@SpringBootApplication**
2. **@Configuration**
3. **@EnableAutoConfiguration**
4. **@ComponentScan**
5. **ApplicationContext**
6. **Inversion of Control (IoC)**
7. **How ApplicationContext Works**
8. **@Component**
9. **@Scope**
10. **Singleton Scope**
11. **Prototype Scope**
12. **@Component – Static and Non-Static Methods**
13. **getBean() – Get Bean by Type**
14. **getBean() – Get Bean by Name**
15. **getBean() – Name + Type**

➤ **@SpringBootApplication:**

@SpringBootApplication is the main annotation used to start a Spring Boot application. It is placed on the main class of the application.

It tells Spring Boot that this is the main configuration class and that the application should be started using Spring Boot's configuration and automatic setup features.

Eg:

```
@SpringBootApplication
public class Application {

    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

➤ **@Configuration:**

@Configuration is used to mark a class as a configuration class. It tells Spring that the class contains configuration information and can be used to define beans.

Eg:

```

@Configuration
public class AppConfig {

    @Bean
    public Student student() {
        return new Student();
    }
}

```

➤ **@EnableAutoConfiguration:**

- @EnableAutoConfiguration tells Spring Boot to automatically configure the application based on the dependencies available in pom.xml.
- This means we do not need to manually configure many things.
- @EnableAutoConfiguration → Automatically configures the application based on dependencies

➤ **@ComponentScan:**

1. @ComponentScan tells Spring to scan packages and find classes that are marked with Spring component annotations.
2. @ComponentScan → Finds Spring components automatically.

The chapter lists:

- @Component
- @Service
- @Repository
- @Controller
- @RestController

➤ **ApplicationContext:**

- ApplicationContext is the Spring container that manages Spring objects called beans.
- Spring creates the objects, stores them inside the container, and provides them whenever they are requested.

```
ApplicationContext context = new AnnotationConfigApplicationContext(AppConfig.class);
```

```
StudentService service = context.getBean(StudentService.class);
```

➤ **Inversion of Control (IoC):**

- Inversion of Control (IoC) means that the control of creating and managing objects is given to the Spring container instead of the programmer.
- StudentService s = context.getBean(StudentService.class);

➤ How ApplicationContext Works

Spring follows these steps:

1.Step 1

Spring starts the container.

2.Step 2

It scans classes containing annotations such as `@Component`.

3.Step 3

Spring creates objects for those classes.

4.Step 4

It stores those objects as beans inside the container.

5.Step 5

When `getBean()` is called, Spring returns the required object.

➤ @Component:

- `@Component` is a Spring annotation used to mark a class as a Spring-managed bean.
- When a class is marked with `@Component`, Spring automatically creates, manages, and injects its object wherever it is required.
- Getting the Bean

```
StudentService service = context.getBean(StudentService.class);
service.display();
```

➤ @Scope:

- `@Scope` is used to define how many objects Spring should create for a bean and how long those objects live inside the Spring container.
- scope controls the **number of bean objects and their lifecycle**.

1. Singleton

2. Prototype

➤ @Component – Static and Non-Static Methods:

- Spring manages **object instances (beans)**.
- A **non-static method** belongs to the bean object, so it can be managed through the Spring bean.
- A **static method** belongs to the class, not to a particular object, so Spring does not manage it as part of the bean instance

Eg:

`@Component`

```
public class StudentService {  
  
    public void display() {  
  
        System.out.println("Non-static method");  
  
    }  
  
    public static void show() {  
  
        System.out.println("Static method");  
  
    }  
  
}
```

Non-static method:

```
StudentService s = context.getBean(StudentService.class);  
  
s.display();
```

Static method:

```
StudentService.show();
```

➤ **getBean() – Get Bean by Type:**

- `getBean()` is used to **retrieve a bean from the IoC container**.
- We can retrieve a bean using its class type.

Eg:

```
StudentService s = context.getBean(StudentService.class);
```

➤ **getBean() – Name + Type**

We can provide both the **bean name and class type**.

Eg:

```
StudentService s = context.getBean("studentService",StudentService.class);
```

This gets the bean using both its name and type.